AB2 ZH – Oracle adatszótár cheat sheet (gyakorlati rész)

ARANYSZABÁLYOK

- Nevek az adatszótárban NAGYBETŰVEL: `owner='NIKOVITS'`, `table_name='EMP'`
- Join mindig **owner + név** (csak névre rossz!)
- SELECT-ben aggregátum nélküli oszlop → legyen a GROUP BY-ban is
- Azonos oszlopnév két nézetben → alias: `c.owner` (különben ORA-00918)
- Sorra vonatkozó feltétel → WHERE; csoportra → HAVING (aliast nem lát!)
- Méret mindig `dba_segments/dba_extents`; `dba_tables.blocks` csak becslés
- „sorrendben" → ORDER BY (különben nincs garantált sorrend)
- Bizonytalan oszlop → `DESC dba_xxx;`

1. KÉRDÉS → NÉZET

tulajdonos, típus, létrehozás	`dba_objects`
van-e tárolása	`dba_objects.data_object_id` (NULL=nincs)
táblatér, particionált, IOT, klaszter	`dba_tables`
oszlop neve, sorszáma, típusa, NULL-e	`dba_tab_columns`
nézet szövege / szinonima / szekvencia	`dba_views` / `dba_synonyms` / `dba_sequences`
adatfájl neve, mérete	`dba_data_files` + `dba_temp_files`
blokkméret, temp-e	`dba_tablespaces`
MÉRET (bytes, blocks, extents)	`dba_segments`
melyik FÁJLBAN, extensenként	`dba_extents` (file_id)
szabad hely	`dba_free_space`
index típusa, melyik táblán	`dba_indexes`
index oszlopai, DESC-e	`dba_ind_columns`
függvény alapú index kifejezése	`dba_ind_expressions`
particionálás típusa, darab	`dba_part_tables`
partíciók, high_value, pozíció	`dba_tab_partitions`
particionáló oszlopok	`dba_part_key_columns`
klaszter típusa (INDEX/HASH)	`dba_clusters`
klaszterkulcs oszlopai	`dba_clu_columns`
egyedi hash függvény	`dba_cluster_hash_expressions`
sor melyik fájl/blokk/sorszám	`DBMS_ROWID` a táblán

Tipp: a feladat [zárójeles] oszlopaiból indulj: `segment_*, bytes, extents` → dba_segments; `column_*` → dba_tab_columns; `file_name` → dba_data_files.

2. NÉZETEK FONTOS OSZLOPAI

dba_objects: owner, object_name, **object_type** (TABLE, VIEW, INDEX, SYNONYM, SEQUENCE, TRIGGER, PROCEDURE, TABLE PARTITION…), subobject_name (partíció), object_id, **data_object_id**, created
dba_tables: owner, table_name, tablespace_name (NULL ha part./IOT), **partitioned** ('YES'/'NO'), **iot_type** ('IOT'/'IOT_OVERFLOW'/NULL), **iot_name** (overflow → eredeti IOT), **cluster_name**, blocks, num_rows (becslés)
dba_tab_columns: owner, table_name, column_name, **column_id**, **data_type** ('NUMBER','VARCHAR2','DATE'), data_length, **nullable** ('Y'/'N') – *nézetek oszlopai is!*
dba_synonyms: owner, synonym_name, table_owner, table_name, db_link · **dba_views**: owner, view_name, text · **dba_sequences**: **sequence_owner**, sequence_name, increment_by, last_number
dba_data_files / dba_temp_files: file_id, file_name, tablespace_name, bytes, blocks · **dba_tablespaces**: tablespace_name, block_size, contents (PERMANENT/TEMPORARY/UNDO)
dba_segments (1 sor = 1 szegmens): owner, segment_name, **partition_name**, **segment_type** (TABLE, INDEX, TABLE PARTITION, TABLE SUBPARTITION, INDEX PARTITION, CLUSTER), tablespace_name, bytes, blocks, extents
dba_extents (1 sor = 1 extens): owner, segment_name, partition_name, segment_type, **file_id**, block_id (első blokk), blocks, bytes · **dba_free_space**: tablespace_name, file_id, block_id, bytes, blocks
dba_indexes: owner, index_name, **table_owner**, table_name, **index_type** (NORMAL, BITMAP, FUNCTION-BASED NORMAL/BITMAP, IOT - TOP, CLUSTER), uniqueness, partitioned
dba_ind_columns: **index_owner**, index_name, table_owner, table_name, column_name, column_position, **descend** ('ASC'/'DESC') ·
dba_ind_expressions: index_owner, index_name, column_expression, column_position
dba_part_tables: owner, table_name, partitioning_type, subpartitioning_type, partition_count · **dba_tab_partitions**: **table_owner**, table_name, partition_name, partition_position, high_value ·
dba_part_key_columns: owner, **name**, object_type, column_name, column_position
dba_clusters: owner, cluster_name, cluster_type, function ·
dba_clu_columns: owner, cluster_name, clu_column_name, table_name, tab_column_name · **dba_cluster_hash_expressions**: owner, cluster_name, hash_expression

3. JOIN-KULCSOK

- tab_columns ↔ tables: **owner + table_name**
- objects ↔ bármi: **owner + object_name** = table/index/segment_name
- segments ↔ tables: owner + **segment_name = table_name**
- segments ↔ indexes: owner + **segment_name = index_name**
- extents / free_space ↔ data_files: **file_id**
- ind_columns ↔ indexes: **index_owner = owner** + index_name
- DBMS_ROWID ↔ data_files: `rowid_relative_fno(rowid) = file_id`

Tulajdonos-oszlop: owner · table_owner (dba_indexes táblája, dba_tab_partitions) · index_owner (dba_ind_*) · sequence_owner

4. KULCSSZÓ → SQL

„van X és Y"	INTERSECT
„de nincs"	MINUS
„vagy"	UNION
„hány" / „hány különböző"	COUNT(*) / COUNT(DISTINCT x)
„…-nként"	GROUP BY
„legalább / több mint N"	GROUP BY + HAVING COUNT(*) >= N
„összesen / összméret"	SUM(bytes)
„legnagyobb / legtöbb"	ORDER BY x DESC FETCH FIRST 1 ROWS WITH TIES
„a két legtöbb"	FETCH FIRST 2 ROWS ONLY
„pontosan N db X típusú" (+más felt.)	HAVING SUM(CASE WHEN … THEN 1 ELSE 0 END) = N
„X-ből >n, Y-ból max m"	két SUM(CASE) a HAVING-ben
„utolsó oszlop X"	MAX(column_id) = MAX(CASE WHEN data_type='X' THEN column_id END)
„első oszlop X"	ugyanez MIN-nel
„1. és 4. oszlop is X"	WHERE column_id IN (1,4) AND data_type='X' + HAVING COUNT(*)=2
„…-val kezdődő / tartalmaz"	LIKE 'Z%' / LIKE '%B%'
„MB-ban"	SUM(bytes)/1024/1024 (ROUND(…,2))
„hány %-ban"	ROUND(100*foglalt/összes, 2)
„hány blokkot FOGLAL"	dba_segments.blocks
„hány blokkban VAN adat"	COUNT(DISTINCT fno||'.'||block) DBMS_ROWID-dal
„csak tábla, nézet ne"	(owner,table_name) IN (SELECT … FROM dba_tables)
„nem particionált tábla szegmense"	segment_type = 'TABLE'
„particionált tábla mérete"	segment_type IN ('TABLE PARTITION','TABLE SUBPARTITION')
két független szám egy sorban	SELECT (SELECT…) a, (SELECT…) b FROM dual

5. ALAPMINTÁK

```
-- A) halmazművelet (balról jobbra hajtódik végre)
SELECT owner FROM dba_objects WHERE object_type='SEQUENCE'
INTERSECT
SELECT owner FROM dba_objects WHERE object_type='SYNONYM'
MINUS
SELECT owner FROM dba_objects WHERE object_type='PROCEDURE';

-- B) "több mint N féle"
SELECT owner FROM dba_objects GROUP BY owner
HAVING COUNT(DISTINCT object_type) > 10;

-- C) "több mint n tábla, max m index" (0 indexes is!)
SELECT owner FROM dba_objects GROUP BY owner
HAVING SUM(CASE WHEN object_type='TABLE' THEN 1 ELSE 0 END) > 20
   AND SUM(CASE WHEN object_type='INDEX' THEN 1 ELSE 0 END) <=
15;

-- D) legtöbb (holtversennyel)
SELECT object_type, COUNT(*) darab FROM dba_objects
GROUP BY object_type ORDER BY darab DESC
FETCH FIRST 1 ROWS WITH TIES;
-- verzió-független: HAVING COUNT(*) =
--   (SELECT MAX(COUNT(*)) FROM dba_objects GROUP BY
object_type)
```

6. OBJEKTUMOK, OSZLOPOK

```
-- DBA_TABLES / DUAL tulajdonosa (SYS) ill. szinonimája (PUBLIC)
SELECT owner, object_name, object_type FROM dba_objects
WHERE object_name IN ('DBA_TABLES','DUAL');
-- tárolást igénylő / nem igénylő típusok (metszet: INTERSECT)
SELECT DISTINCT object_type FROM dba_objects WHERE
data_object_id IS NOT NULL;
SELECT DISTINCT object_type FROM dba_objects WHERE
data_object_id IS NULL;
-- GYAK01
CREATE TABLE gyak01 AS SELECT table_name FROM dba_tables
WHERE owner='NIKOVITS' AND table_name LIKE '%B%';

-- oszlopszám / 6. oszlop típusa / NULL-t engedő oszlopok
SELECT COUNT(*) FROM dba_tab_columns WHERE owner='NIKOVITS' AND
table_name='EMP';
SELECT data_type FROM dba_tab_columns
WHERE owner='NIKOVITS' AND table_name='EMP' AND column_id=6;
SELECT column_name, column_id FROM dba_tab_columns
WHERE owner='NIKOVITS' AND table_name='EMP' AND nullable='Y'
ORDER BY column_id;
-- 'Z'-vel kezdődő oszlop / legalább 8 DATE
SELECT DISTINCT owner, table_name FROM dba_tab_columns WHERE
column_name LIKE 'Z%';
SELECT owner, table_name FROM dba_tab_columns WHERE
data_type='DATE'
GROUP BY owner, table_name HAVING COUNT(*) >= 8;
-- 1. és 4. oszlop is VARCHAR2
SELECT owner, table_name FROM dba_tab_columns
WHERE column_id IN (1,4) AND data_type='VARCHAR2'
GROUP BY owner, table_name HAVING COUNT(*) = 2;
-- csak TÁBLA, pontosan 2 NUMBER, utolsó oszlop DATE
SELECT c.owner, c.table_name FROM dba_tab_columns c
WHERE (c.owner, c.table_name) IN (SELECT owner, table_name FROM
dba_tables)
GROUP BY c.owner, c.table_name
HAVING SUM(CASE WHEN c.data_type='NUMBER' THEN 1 ELSE 0 END) = 2
   AND MAX(c.column_id) = MAX(CASE WHEN c.data_type='DATE' THEN
c.column_id END);
```

7. SZINONIMA, NÉZET, SZEKVENCIA, DB LINK

```
-- SELECT * FROM sz1; -> mit kérdeztünk le? (láncot követni!)
SELECT owner, synonym_name, table_owner, table_name, db_link
FROM dba_synonyms WHERE synonym_name='SZ1';
SELECT object_type FROM dba_objects WHERE owner='XY' AND
object_name='ABC';
SELECT text FROM dba_views WHERE owner='XY' AND view_name='ABC';

CREATE SEQUENCE oszt_seq START WITH 10 INCREMENT BY 10;
INSERT INTO osztaly(oazon, nev) VALUES (oszt_seq.NEXTVAL, 'IT');
INSERT INTO dolgozo(dazon, nev, oazon) VALUES (1, 'Kiss',
oszt_seq.CURRVAL);
UPDATE osztaly SET oazon = oszt_seq.NEXTVAL WHERE oazon = 10;
UPDATE dolgozo SET oazon = oszt_seq.CURRVAL WHERE oazon = 10;

CREATE DATABASE LINK aramisdb CONNECT TO felh IDENTIFIED BY
jelszo
USING 'aramis.inf.elte.hu:1521/aramis';
-- Mekong országai folyásirányban (oszlopnév: DESC
vilag_orszagai)
SELECT v.orszag FROM nikovits.vilag_orszagai v,
nikovits.folyok@aramisdb f
WHERE f.nev = 'Mekong'
  AND ','||f.orszagok||',' LIKE '%,'||v.tld||',%'
ORDER BY INSTR(f.orszagok, v.tld);
```

8. TÁROLÁS

Hierarchia: táblatér → adatfájl → szegmens → extens → blokk

```
-- fájlok méret szerint (temp is)
SELECT file_name, bytes FROM dba_data_files
UNION ALL SELECT file_name, bytes FROM dba_temp_files ORDER BY
bytes DESC;
-- táblaterek: fájlszám, összméret
SELECT tablespace_name, COUNT(*), SUM(bytes) FROM (
  SELECT tablespace_name, bytes FROM dba_data_files UNION ALL
  SELECT tablespace_name, bytes FROM dba_temp_files) GROUP BY
tablespace_name;
-- blokkméret / adatfájl nélküli táblatér (temp)
SELECT block_size FROM dba_tablespaces WHERE
tablespace_name='USERS';
SELECT tablespace_name FROM dba_tablespaces MINUS SELECT
tablespace_name FROM dba_data_files;
-- táblaterenként szegmensek, MB, csak >100MB
SELECT tablespace_name, COUNT(*) darab, SUM(bytes)/1024/1024
meret_mb
FROM dba_segments GROUP BY tablespace_name HAVING
SUM(bytes)/1024/1024 > 100;
-- legnagyobb tábla szegmens + extensek (INDEX:
segment_type='INDEX', blocks)
SELECT owner, segment_name, extents FROM dba_segments WHERE
segment_type='TABLE'
ORDER BY bytes DESC FETCH FIRST 1 ROWS WITH TIES;
-- NIKOVITS legtöbb extensből álló táblája
SELECT segment_name, extents FROM dba_segments
WHERE owner='NIKOVITS' AND segment_type='TABLE'
ORDER BY extents DESC FETCH FIRST 1 ROWS WITH TIES;
-- adatfájlonként foglalt hely
SELECT f.file_name, f.bytes, NVL(SUM(e.bytes),0) foglalt
FROM dba_data_files f LEFT JOIN dba_extents e ON e.file_id =
f.file_id
GROUP BY f.file_name, f.bytes;
-- NIKOVITS foglalása fájlonként
SELECT f.file_name, SUM(e.bytes) foglalt_byte
FROM dba_extents e JOIN dba_data_files f ON f.file_id =
e.file_id
WHERE e.owner='NIKOVITS' GROUP BY f.file_name ORDER BY
foglalt_byte DESC;
-- két legtöbb helyet foglaló felhasználó
SELECT owner, SUM(bytes) FROM dba_segments GROUP BY owner
ORDER BY 2 DESC FETCH FIRST 2 ROWS ONLY;
-- users02.dbf: extensek, szabad terület, %
SELECT COUNT(*), SUM(e.bytes) FROM dba_extents e
  JOIN dba_data_files f ON e.file_id=f.file_id WHERE f.file_name
LIKE '%users02.dbf';
SELECT COUNT(*), SUM(s.bytes) FROM dba_free_space s
  JOIN dba_data_files f ON s.file_id=f.file_id WHERE f.file_name
LIKE '%users02.dbf';
SELECT ROUND(100*(SELECT SUM(e.bytes) FROM dba_extents e
   WHERE e.file_id=f.file_id)/f.bytes, 2)
FROM dba_data_files f WHERE f.file_name LIKE '%users02.dbf';
-- több fájlban lévő táblák / melyik fájlokban
SELECT segment_name FROM dba_extents WHERE owner='NIKOVITS' AND
segment_type='TABLE'
GROUP BY segment_name HAVING COUNT(DISTINCT file_id) > 1;
SELECT DISTINCT f.file_name FROM dba_extents e JOIN
dba_data_files f
  ON e.file_id=f.file_id WHERE e.owner='NIKOVITS' AND
e.segment_name='TABLA_123';
-- melyik táblatéren (particionáltnál NULL ->
dba_tab_partitions)
SELECT tablespace_name FROM dba_tables WHERE owner='ORAUSER' AND
table_name='DOLGOZO';
SELECT partition_name, tablespace_name FROM dba_tab_partitions
WHERE table_owner='NIKOVITS' AND table_name='ELADASOK';
```

9. ROWID, BLOKKOK

ROWID = `OOOOOO FFF BBBBBB RRR` (adatobj.-id, rel. fájl, blokk, sor).
Függvények: `dbms_rowid.rowid_object / rowid_relative_fno / rowid_block_number / rowid_row_number (rowid)`

```
-- hány blokkot FOGLAL / hány blokkban VAN adat
SELECT blocks FROM dba_segments
WHERE owner='NIKOVITS' AND segment_name='CIKK' AND
segment_type='TABLE';
SELECT COUNT(DISTINCT
dbms_rowid.rowid_relative_fno(rowid)||'.'||
                    dbms_rowid.rowid_block_number(rowid)) FROM
nikovits.cikk;
-- egy sorban a kettő
SELECT (SELECT blocks FROM dba_segments WHERE owner='NIKOVITS'
         AND segment_name='EMP' AND segment_type='TABLE')
szegmens_blokk,
       (SELECT COUNT(DISTINCT
dbms_rowid.rowid_relative_fno(rowid)||'.'||
              dbms_rowid.rowid_block_number(rowid)) FROM
nikovits.emp) adat_blokk
FROM dual;
-- blokkonként sorok száma
SELECT dbms_rowid.rowid_relative_fno(rowid) file_id,
       dbms_rowid.rowid_block_number(rowid) blokk_id, COUNT(*)
FROM nikovits.cikk
GROUP BY dbms_rowid.rowid_relative_fno(rowid),
dbms_rowid.rowid_block_number(rowid)
ORDER BY 1, 2;
-- egy sor: fájl, blokk, sorszám
SELECT f.file_name, dbms_rowid.rowid_block_number(e.rowid),
       dbms_rowid.rowid_row_number(e.rowid)
FROM nikovits.eladasok e JOIN dba_data_files f
  ON f.file_id = dbms_rowid.rowid_relative_fno(e.rowid)
WHERE e.szla_szam = 100;
-- melyik adatobjektumban (partícióban) van
SELECT object_name, object_type, subobject_name, object_id,
data_object_id
FROM dba_objects WHERE data_object_id =
 (SELECT dbms_rowid.rowid_object(rowid) FROM nikovits.eladasok
WHERE szla_szam=100);
-- 128K tábla + extra extens
CREATE TABLE cikk2 SEGMENT CREATION IMMEDIATE TABLESPACE example
  STORAGE (INITIAL 128K) PCTFREE 10 AS SELECT * FROM
nikovits.cikk WHERE 1=0;
ALTER TABLE cikk2 ALLOCATE EXTENT (SIZE 128K);
-- constraintet sértő sorok
CREATE TABLE exceptions(row_id ROWID, owner VARCHAR2(30),
  table_name VARCHAR2(30), constraint VARCHAR2(30));
ALTER TABLE t ADD CONSTRAINT t_uq UNIQUE(o) EXCEPTIONS INTO
exceptions;
SELECT * FROM t WHERE rowid IN (SELECT row_id FROM exceptions);
```

10. INDEXEK

```
-- csökkenő sorrendű indexelt oszlop (furcsa név ->
dba_ind_expressions)
SELECT DISTINCT table_owner, table_name FROM dba_ind_columns
WHERE descend='DESC';
-- legalább 9 oszlopos index
SELECT index_owner, index_name FROM dba_ind_columns
GROUP BY index_owner, index_name HAVING COUNT(*) >= 9;
-- CUSTOMERS bitmap indexei
SELECT index_name FROM dba_indexes
WHERE table_owner='NIKOVITS' AND table_name='CUSTOMERS' AND
index_type='BITMAP';
-- legalább 2 bitmap indexes táblák
SELECT table_owner, table_name, COUNT(*) FROM dba_indexes
WHERE index_type LIKE '%BITMAP' GROUP BY table_owner, table_name
HAVING COUNT(*) >= 2;
-- kétoszlopos, függvény alapú kifejezéssel
SELECT index_owner, index_name FROM dba_ind_columns
GROUP BY index_owner, index_name HAVING COUNT(*) = 2
INTERSECT SELECT index_owner, index_name FROM
dba_ind_expressions;
SELECT index_name, column_expression FROM dba_ind_expressions
WHERE index_owner='NIKOVITS';
-- tábla indexei oszlopokkal, sorrendben
SELECT index_name, column_name, column_position FROM
dba_ind_columns
WHERE table_owner='NIKOVITS' AND table_name='CUSTOMERS'
ORDER BY index_name, column_position;
-- indexek mérete
SELECT i.index_name, NVL(SUM(s.bytes),0) FROM dba_indexes i
LEFT JOIN dba_segments s ON s.owner=i.owner AND
s.segment_name=i.index_name
WHERE i.table_owner='NIKOVITS' AND i.table_name='CUSTOMERS'
GROUP BY i.index_name ORDER BY i.index_name;
```

11. IOT (index-szervezett tábla)

Az IOT táblának NINCS szegmense (tablespace NULL). Adat az 'IOT - TOP' index szegmensében. Túlcsordulás: külön tábla, iot_type='IOT_OVERFLOW', iot_name = eredeti.

```
SELECT table_name, tablespace_name FROM dba_tables
WHERE owner='NIKOVITS' AND iot_type='IOT';
SELECT table_name, index_name, tablespace_name FROM dba_indexes
WHERE owner='NIKOVITS' AND index_type='IOT - TOP';
SELECT segment_name, segment_type, bytes FROM dba_segments
WHERE owner='NIKOVITS' AND segment_name IN
 (SELECT index_name FROM dba_indexes WHERE owner='NIKOVITS' AND
index_type='IOT - TOP');
-- túlcsordulási részek / méretük / melyik IOT-nak van
SELECT table_name, iot_name FROM dba_tables WHERE
owner='NIKOVITS' AND iot_type='IOT_OVERFLOW';
SELECT owner, iot_name table_name FROM dba_tables WHERE
iot_type='IOT_OVERFLOW';
-- CIKK_IOT minden része
SELECT object_name, object_type, object_id, data_object_id FROM
dba_objects
WHERE owner='NIKOVITS' AND (object_name='CIKK_IOT'
 OR object_name IN (SELECT index_name FROM dba_indexes
                    WHERE owner='NIKOVITS' AND
table_name='CIKK_IOT')
 OR object_name IN (SELECT table_name FROM dba_tables
                    WHERE owner='NIKOVITS' AND
iot_name='CIKK_IOT'));
-- IOT-k pontosan 1 DATE oszloppal
SELECT owner, table_name FROM dba_tab_columns WHERE
data_type='DATE'
 AND (owner, table_name) IN (SELECT owner, table_name FROM
dba_tables WHERE iot_type='IOT')
GROUP BY owner, table_name HAVING COUNT(*) = 1;
-- IOT mérete: index szegmens + overflow szegmens
SELECT SUM(bytes) FROM dba_segments WHERE owner='NIKOVITS' AND (
 segment_name IN (SELECT index_name FROM dba_indexes
                  WHERE owner='NIKOVITS' AND
table_name='CIKK_IOT')
 OR segment_name IN (SELECT table_name FROM dba_tables
                     WHERE owner='NIKOVITS' AND
iot_name='CIKK_IOT'));
```

12. PARTÍCIÓK

A particionált tábla logikai (nincs szegmense), a partíció a tárolt; alpartíciónál csak az alpartíció. dba_segments.segment_name = TÁBLA neve, partition_name = partíció.

```
SELECT table_name, partitioning_type, subpartitioning_type
FROM dba_part_tables WHERE owner='NIKOVITS';
-- partíciók blokkjai (pontos: szegmensből)
SELECT partition_name, blocks FROM dba_segments
WHERE owner='NIKOVITS' AND segment_name='ELADASOK';
-- particionáló oszlopok
SELECT column_name, column_position FROM dba_part_key_columns
WHERE owner='NIKOVITS' AND name='ELADASOK' AND
object_type='TABLE';
-- 2. partíció értékei
SELECT partition_name, high_value FROM dba_tab_partitions
WHERE table_owner='NIKOVITS' AND table_name='ELADASOK3' AND
partition_position=2;
-- logikai + fizikai részek
SELECT object_name, object_type, subobject_name, object_id,
data_object_id
FROM dba_objects WHERE owner='NIKOVITS' AND
object_name='ELADASOK4';
SELECT segment_name, segment_type, partition_name, bytes
FROM dba_segments WHERE owner='NIKOVITS' AND
segment_name='ELADASOK4';
-- legalább 4 partíció
SELECT table_owner, table_name, COUNT(*) FROM dba_tab_partitions
GROUP BY table_owner, table_name HAVING COUNT(*) >= 4;
-- particionált táblák mérete (legnagyobb: ORDER BY + FETCH)
SELECT owner, segment_name, ROUND(SUM(bytes)/1024/1024,2)
meret_mb
FROM dba_segments WHERE segment_type IN ('TABLE
PARTITION','TABLE SUBPARTITION')
GROUP BY owner, segment_name ORDER BY meret_mb DESC;
```

13. KLASZTEREK

```
CREATE CLUSTER dolg_oszt (oazon NUMBER(2));
CREATE INDEX dolg_oszt_idx ON CLUSTER dolg_oszt;
CREATE TABLE osztaly (oazon NUMBER(2), nev VARCHAR2(20)) CLUSTER
dolg_oszt(oazon);
CREATE TABLE dolgozo (dazon NUMBER, nev VARCHAR2(20), oazon
NUMBER(2))
  CLUSTER dolg_oszt(oazon);
-- hash klaszter: azonos kulcs -> azonos blokk
CREATE CLUSTER hc (kulcs NUMBER) HASHKEYS 100 HASH IS kulcs;
-- tábla nélküli klaszter
SELECT owner, cluster_name FROM dba_clusters
MINUS SELECT owner, cluster_name FROM dba_tables WHERE
cluster_name IS NOT NULL;
-- pontosan 2 tábla rajta
SELECT owner, cluster_name FROM dba_tables WHERE cluster_name IS
NOT NULL
GROUP BY owner, cluster_name HAVING COUNT(*) = 2;
-- 3 oszlopos klaszterkulcs (DISTINCT!)
SELECT owner, cluster_name FROM dba_clu_columns
GROUP BY owner, cluster_name HAVING COUNT(DISTINCT
clu_column_name) = 3;
-- nem alapértelmezett hash függvény
SELECT COUNT(*) FROM dba_cluster_hash_expressions;
-- klaszteres táblák + típus
SELECT t.owner, t.cluster_name, c.cluster_type, t.table_name
FROM dba_tables t JOIN dba_clusters c
  ON c.owner=t.owner AND c.cluster_name=t.cluster_name;
-- sima (heap) táblák száma
SELECT COUNT(*) FROM dba_tables WHERE owner='NIKOVITS'
  AND partitioned='NO' AND iot_type IS NULL AND cluster_name IS
NULL;
-- tárolási mód
SELECT owner, table_name, cluster_name, partitioned, iot_type
FROM dba_tables
WHERE owner='NIKOVITS' AND table_name IN
('EMP','ELADASOK5','CIKK_IOT','EMP_CLT');
```

UTOLSÓ ELLENŐRZÉS

- Minden „és” → INTERSECT, minden „nincs” → MINUS megvan?
- Nagybetű? Alias közös oszlopnál? GROUP BY ↔ SELECT egyezik?
- Particionált / IOT → tablespace NULL; temp fájlok külön nézetben
- `NOT IN` + NULL az alkérdésben → üres! Inkább MINUS / NOT EXISTS